QUESTION NO: 1
You need to destroy all of the resources in your Terraform workspace, except for aws_instance.ubuntu[1], which you want to keep. How can you tell Terraform to stop managing that specific resource without destroying it?
A. Remove the resource block from your configuration.
B. Change the value of the count argument on the resource.
C. Run terraform state rm aws_instance.ubuntu[1].
D. Use a moved block.
Answer: C
Explanation:
Rationale for Correct answer: terraform state rm <address> removes a resource from Terraform state without destroying the real infrastructure. After removal, Terraform "forgets" it and will no longer manage it (unless it's re-imported). That matches the requirement: keep the resource, stop managing it.
Analysis of Incorrect Options (Distractors):
A: Removing the resource block causes Terraform to plan to destroy the resource (because it's in state but no longer in config), which is the opposite of "keep it." B: Changing count can cause Terraform to destroy instances that no longer have an address (depending on indexing), and it doesn't explicitly "stop managing" a specific existing instance safely.
D: moved blocks are for renaming/refactoring addresses while keeping management; they do not stop managing a resource.
Key Concept: Detaching resources from Terraform management using state subcommands (terraform state rm).
Reference:

QUESTION NO: 2
How is terraform import run?
A. As a part of terraform init
B. As a part of terraform plan
C. As a part of terraform refresh
D. By an explicit call
E. All of the above
Answer: D
Explanation:
The terraform import command is not part of any other Terraform workflow. It must be explicitly invoked by the user with the appropriate arguments, such as the resource address and the ID of the existing infrastructure to import. Reference = [Importing Infrastructure]

QUESTION NO: 3
Which of the following ate advantages of using infrastructure as code (IaC) instead of provisioning with a graphical user interface (GUI)? Choose two correct answers.
A. Lets you version, reuse, and share infrastructure configuration
B. Provisions the same resources at a lower cost
C. Secures your credentials

D. Reduces risk of operator error

E. Prevents manual modifications to your resources

Answer: A,D

Explanation:

It lets you version, reuse, and share infrastructure configuration as code files, which can be stored in a source control system and integrated with your CI/CD pipeline.

It reduces risk of operator error by automating repetitive tasks and ensuring consistency across environments. IaC does not necessarily provision resources at a lower cost, secure your credentials, or prevent manual modifications to your resources - these depend on other factors such as your cloud provider, your security practices, and your access policies.

QUESTION NO: 4

You have a simple Terraform configuration containing one virtual machine (VM) in a cloud provider. You run terraform apply and the VM is created successfully. What will happen if you terraform apply again immediately afterward without changing any Terraform code?

A. Terraform will terminate and recreate the VM.

B. Terraform will create another duplicate VM.

C. Terraform will apply the VM to the state file.

D. Nothing

Answer: D

Explanation:

Terraform follows adeclarativeapproach, meaning it ensures the infrastructure matches the configuration.

If you run terraform apply againwithout any changes, Terraform willdetect that the infrastructure is already up to dateand willdo nothing.

The command will complete successfully with the message"No changes. Your infrastructure matches the configuration." Official Terraform Documentation Reference:

Terraform Apply - HashiCorp Documentation

QUESTION NO: 5

The -refresh-only parameter will update your state file when used with terraform plan.

A. True

B. False

Answer: B

Explanation:

Rationale for Correct answer: terraform plan -refresh-only refreshes (reconciles) the state with the real infrastructure only for the purpose of the plan output. It does not persist (write) those updates to the state file. To actually update the stored state, you must run terraform apply -refresh-only (or a normal terraform apply). This matches the Terraform CLI objective: plan previews changes; apply makes changes (including writing state).

Analysis of Incorrect Options (Distractors):

A (True): Incorrect because terraform plan does not write state; it only calculates and displays what would change.

Key Concept: The plan vs apply workflow and when Terraform writes state.

QUESTION NO: 6
Terraformrequiresthe Go runtime as a prerequisite for installation.
A. True
B. False
Answer: B
Explanation:
Terraformis written in Go, but it isdistributed as a standalone binaryanddoes notrequire the Go runtime.
Users do not need to install Go to run Terraform.
Official Terraform Documentation Reference:
Terraform Install Guide

QUESTION NO: 7
You should run terraform fnt to rewrite all Terraform configurations within the current working directory to conform to Terraform-style conventions.
A. True
B. False
Answer: A
Explanation:
You should run terraform fmt to rewrite all Terraform configurations within the current working directory to conform to Terraform-style conventions. This command applies a subset of the Terraform language style conventions, along with other minor adjustments for readability. It is recommended to use this command to ensure consistency of style across different Terraform codebases. The command is optional, opinionated, and has no customization options, but it can help you and your team understand the code more quickly and easily. Reference = : Command: fmt : Using Terraform fmt Command to Format Your Terraform Code

QUESTION NO: 8
Which of the following isnotan advantage of using Infrastructure as Code (IaC) operations?
A. Self-service infrastructure deployment.
B. Modify a count parameter to scale resources.
C. API-driven workflows.
D. Troubleshoot via a Linux diff command.
E. Public cloud console configuration workflows.
Answer: E
Explanation:
E (Correct)-IaC aims to eliminate manual cloud console workflowsby using code-based automation.
A, B, C (Advantages of IaC)- IaC enablesself-service deployment, API-driven workflows, and dynamic resource scaling.
D (diff command)- While useful, troubleshooting via diff is not acore advantageof IaC.
Official Terraform Documentation Reference:
What is Infrastructure as Code?

QUESTION NO: 9
Which of these ate secure options for storing secrets for connecting to a Terraform remote backend? Choose two correct answers.
A. A variable file
B. Defined in Environment variables
C. Inside the backend block within the Terraform configuration
D. Defined in a connection configuration outside of Terraform
Answer: B,D
Explanation:
Environment variables and connection configurations outside of Terraform are secure options for storing secrets for connecting to a Terraform remote backend. Environment variables can be used to set values for input variables that contain secrets, such as backend access keys or tokens. Terraform will read environment variables that start with TF_VAR_ and match the name of an input variable. For example, if you have an input variable called backend_token, you can set its value with the environment variable TF_VAR_backend_token1. Connection configurations outside of Terraform are files or scripts that provide credentials or other information for Terraform to connect to a remote backend. For example, you can use a credentials file for the S3 backend2, or a shell script for the HTTP backend3. These files or scripts are not part of the Terraform configuration and can be stored securely in a separate location. The other options are not secure for storing secrets. A variable file is a file that contains values for input variables. Variable files are usually stored in the same directory as the Terraform configuration or in a version control system. This exposes the secrets to anyone who can access the files or the repository. You should not store secrets in variable files1. Inside the backend block within the Terraform configuration is where you specify the type and settings of the remote backend. The backend block is part of the Terraform configuration and is usually stored in a version control system. This exposes the secrets to anyone who can access the configuration or the repository. You should not store secrets in the backend block4. Reference = [Terraform Input Variables]1, [Backend Type: s3]2, [Backend Type: http]3, [Backend Configuration]4

QUESTION NO: 10
Why would you use the -replace flag for terraform apply?
A. You want Terraform to ignore a resource on the next apply
B. You want Terraform to destroy all the infrastructure in your workspace
C. You want to force Terraform to destroy a resource on the next apply
D. You want to force Terraform to destroy and recreate a resource on the next apply
Answer: D
Explanation:
The -replace flag is used with the terraform apply command when there is a need to explicitly force Terraform to destroy and then recreate a specific resource during the next apply. This can be necessary in situations where a simple update is insufficient or when a resource must be re-provisioned to pick up certain changes.

QUESTION NO: 11

Which of the following is not a benefit of adopting infrastructure as code?

A. Versioning

B. A Graphical User Interface

C. Reusability of code

D. Automation

Answer: B

Explanation:

Infrastructure as Code (IaC) provides several benefits, including the ability to version control infrastructure, reuse code, and automate infrastructure management. However, IaC is typically associated with declarative configuration files and does not inherently provide a graphical user interface (GUI). A GUI is a feature that may be provided by specific tools or platforms built on top of IaC principles but is not a direct benefit of IaC itself1.

Reference = The benefits of IaC can be verified from the official HashiCorp documentation on "What is Infrastructure as Code with Terraform?" provided by HashiCorp Developer1.

QUESTION NO: 12

Which method for sharing Terraform modules fulfills the following criteria:

Keeps the module configurations confidential within your organization.

Supports Terraform's semantic version constraints.

Provides a browsable directory of your modules.

A. A Git repository containing your modules.

B. Public Terraform module registry.

C. A subfolder within your workspace.

D. HCP Terraform/Terraform Cloud private registry.

Answer: D

Explanation:

Confidentiality: UsingHCP Terraform/Terraform Cloud's private registrykeeps the module configurations within your organization, ensuring privacy and access control.

Browsable Directory: The private registry offers a user interface to browse modules, making it easy for users within the organization to locate and manage modules.

This setup aligns with HashiCorp's design forprivate registry supportin Terraform, meeting all listed requirements for secure, version-controlled, and searchable module storage.

QUESTION NO: 13

Exhibit:

Error: Saved plan is stale

The given plan file can no longer be applied because the state was changed by another operation after the plan was created.

You have a saved execution plan containing desired changes for infrastructure managed by Terraform. After running terraform apply my.tfplan, you receive the error shown. How can you apply the desired changes? (Pick the 2 correct responses below.)

A. Generate a new execution plan file with terraform plan, and apply the new plan.

B. Run terraform apply without the saved execution plan.

C. Force the apply command by adding the flag -lock=false.

D. Refresh the current state data using the -refresh-only flag.

E. Update the current plan file using the terraform state push command.

Answer: A,B

Explanation:

Rationale for Correct answer:

A: The plan is stale because state changed after it was created. The correct fix is to recreate the plan against the latest state and apply that new plan.

B: Running terraform apply without specifying the old plan causes Terraform to re-plan using the current state and configuration, then apply the resulting changes.

Analysis of Incorrect Options (Distractors):

C: -lock=false disables locking; it does not make an old plan valid and is unsafe in team environments.

D: -refresh-only changes what's recorded in state (when applied) but does not "unstale" an existing saved plan file.

E: terraform state push is for advanced/manual state management and does not update a saved plan file; using it here is inappropriate and risky.

Key Concept: Saved plans are tied to a specific state snapshot; if state changes, you must re-plan.

Reference:

QUESTION NO: 14

Which of the following should you add in the required_providers block to define a provider version constraint?

A. version ~> 3.1

B. version >= 3.1

C. version = ">= 3.1"

Answer: C

Explanation:

Rationale for Correct Answer (C):

This ensures compatibility with Terraform's syntax.

Analysis of Incorrect Options:

A, B: Incorrect syntax, missing quotes or wrong operator format.

C: Correct Terraform syntax.

Key Concept:

Correct syntax in provider version constraints ensures reproducibility.

Reference:

Terraform Exam Objective - Manage Terraform Resources and Providers.

QUESTION NO: 15

What are some benefits of using Sentinel with Terraform Cloud/Terraform Cloud? Choose three correct answers.

A. You can restrict specific resource configurations, such as disallowing the use of CIDR=0.0.0.0/0.

B. You can check out and check in cloud access keys

C. Sentinel Policies can be written in HashiCorp Configuration Language (HCL)

D. Policy-as-code can enforce security best practices

E. You can enforce a list of approved AWS AMIs

Answer: A,D,E

Explanation:

Sentinel is a policy-as-code framework that allows you to define and enforce rules on your Terraform configurations, states, and plans1. Some of the benefits of using Sentinel with Terraform Cloud/Terraform Enterprise are:

* You can restrict specific resource configurations, such as disallowing the use of CIDR=0.0.0.0/0, which would open up your network to the entire internet. This can help you prevent misconfigurations or security vulnerabilities in your infrastructure2.
* Policy-as-code can enforce security best practices, such as requiring encryption, authentication, or compliance standards. This can help you protect your data and meet regulatory requirements3.
* You can enforce a list of approved AWS AMIs, which are pre-configured images that contain the operating system and software you need to run your applications. This can help you ensure consistency, reliability, and performance across your infrastructure4.

Reference =

* 1: Terraform and Sentinel | Sentinel | HashiCorp Developer
* 2: Terraform Learning Resources: Getting Started with Sentinel in Terraform Cloud
* 3: Exploring the Power of HashiCorp Terraform, Sentinel, Terraform Cloud ...
* 4: Using New Sentinel Features in Terraform Cloud - Medium

QUESTION NO: 16

Your risk management organization requires that new AWS S3 buckets must be private and encrypted at rest. How can Terraform Cloud automatically and proactively enforce this security control?

A. Auditing cloud storage buckets with a vulnerability scanning tool

B. By adding variables to each Terraform Cloud workspace to ensure these settings are always enabled

C. With an S3 module with proper settings for buckets

D. With a Sentinel policy, which runs before every apply

Answer: D

Explanation:

The best way to automatically and proactively enforce the security control that new AWS S3 buckets must be private and encrypted at rest is with a Sentinel policy, which runs before every apply. Sentinel is a policy as code framework that allows you to define and enforce logic-based policies for your infrastructure. Terraform Cloud supports Sentinel policies for all paid tiers, and can run them before any terraform plan or terraform apply operation. You can write a Sentinel policy that checks the configuration of the S3 buckets and ensures that they have the proper settings for privacy and encryption, and then assign the policy to your Terraform Cloud organization or workspace. This way, Terraform Cloud will prevent any changes that violate the policy from being applied. Reference = [Sentinel Policy Framework], [Manage Policies in Terraform Cloud], [Write and Test Sentinel Policies for Terraform]

QUESTION NO: 17

You have multiple team members collaborating on infrastructure as code (IaC) using Terraform, and want to apply formatting standards for readability.
How can you format Terraform HCL (HashiCorp Configuration Language) code according to standard Terraform style convention?

A. Run the terraform fmt command during the code linting phase of your CI/CD process Most Voted

B. Designate one person in each team to review and format everyone's code

C. Manually apply two spaces indentation and align equal sign "=" characters in every Terraform file (*.tf)

D. Write a shell script to transform Terraform files using tools such as AWK, Python, and sed

Answer: A

Explanation:
The terraform fmt command is used to rewrite Terraform configuration files to a canonical format and style. This command applies a subset of the Terraform language style conventions, along with other minor adjustments for readability. Running this command on your configuration files before committing them to source control can help ensure consistency of style between different Terraform codebases, and can also make diffs easier to read. You can also use the -check and -diff options to check if the files are formatted and display the formatting changes respectively2. Running the terraform fmt command during the code linting phase of your CI/CD process can help automate this process and enforce the formatting standards for your team. Reference = [Command: fmt]2

QUESTION NO: 18
You want to define a single input variable to capture configuration values for a server. The values must represent memory as a number, and the server name as a string.
Which variable type could you use for this input?

A. List

B. Object

C. Map

D. Terraform does not support complex input variables of different types

Answer: B

Explanation:
This is the variable type that you could use for this input, as it can store multiple attributes of different types within a single value. The other options are either invalid or incorrect for this use case.

QUESTION NO: 19
You are working on some new application features and you want to spin up a copy of your production deployment to perform some quick tests. In order to avoid having to configure a new state backend, what open source Terraform feature would allow you create multiple states but still be associated with your current code?

A. Terraform data sources

B. Terraform local values

C. Terraform modules

D. Terraform workspaces
E. None of the above
Answer: D
Explanation:
Terraform workspaces allow you to create multiple states but still be associated with your current code. Workspaces are like "environments" (e.g. staging, production) for the same configuration. You can use workspaces to spin up a copy of your production deployment for testing purposes without having to configure a new state backend. Terraform data sources, local values, and modules are not features that allow you to create multiple states. Reference = Workspaces and How to Use Terraform Workspaces

QUESTION NO: 20
Which of these actions are forbidden when the Terraform state file is locked? (Pick the 3 correct responses)
A. terraform apply
B. terraform state list
C. terraform destroy
D. terraform fmt
Answer: A,B,C
Explanation:
When the state file is locked, operations that modify or depend on the state (liketerraform apply,terraform destroy, andterraform state list) are blocked.terraform fmtonly formats the configuration files and does not interact with the state, so it is allowed.
Reference:
Terraform State Locking

QUESTION NO: 21
A senior admin accidentally deleted some of your cloud instances. What will Terraform do when you run terraform apply?
A. Tear down the entire workspace's infrastructure and rebuild it.
B. Build a completely brand new set of infrastructure.
C. Rebuild only the instances that were deleted.
D. Stop and generate an error message about the missing instances.
Answer: C
Explanation:
Terraform detects infrastructure drift by comparing thestate filewith the actual infrastructure. When an instance ismanually deleted, Terraformsees it as missingand marks it for recreation .
Running terraform apply willonly recreate the missing instanceswhile leaving the rest of the infrastructure unchanged.
Explanation of incorrect answers:
A (Tear down everything and rebuild)- Incorrect. Terraform does not destroy existing infrastructure unless explicitly told to.
B (Build a completely new set of infrastructure)- Incorrect. Terraform does not create

duplicates unless configuration changes.
D (Stop and error out)- Incorrect. Terraform does not fail; itrebuilds missing resourcesautomatically.
Official Terraform Documentation Reference:
Handling Infrastructure Drift

QUESTION NO: 22
You ate making changes to existing Terraform code to add some new infrastructure. When is the best time to run terraform validate?
A. After you run terraform apply so you can validate your infrastructure
B. Before you run terraform apply so you can validate your provider credentials
C. Before you run terraform plan so you can validate your code syntax
D. After you run terraform plan so you can validate that your state file is consistent with your infrastructure
Answer: C
Explanation:
This is the best time to run terraform validate, as it will check your code for syntax errors, typos, and missing arguments before you attempt to create a plan. The other options are either incorrect or unnecessary.

QUESTION NO: 23
Which of the following module source paths does not specify a remote module?
A. Source = "module/consul"
B. Source = "githhub.comicrop/example"
C. Source ="git@github.com:hasicrop/example.git"
D. Source = "hasicrop/consul/aws"
Answer: A
Explanation:
The module source path that does not specify a remote module is source = "module/consul". This specifies a local module, which is a module that is stored in a subdirectory of the current working directory. The other options are all examples of remote modules, which are modules that are stored outside of the current working directory and can be accessed by various protocols, such as Git, HTTP, or the Terraform Registry. Remote modules are useful for sharing and reusing code across different configurations and environments. Reference = [Module Sources], [Local Paths], [Terraform Registry], [Generic Git Repository], [GitHub]

QUESTION NO: 24
You add a new provider to your configuration and immediately run terraform apply in the CD using the local backend. Why does the apply fail?
A. The Terraform CD needs you to log into Terraform Cloud first
B. Terraform requires you to manually run terraform plan first
C. Terraform needs to install the necessary plugins first
D. Terraform needs you to format your code according to best practices first
Answer: C

Explanation:
The reason why the apply fails after adding a new provider to the configuration and immediately running terraform apply in the CD using the local backend is because Terraform needs to install the necessary plugins first. Terraform providers are plugins that Terraform uses to interact with various cloud services and other APIs. Each provider has a source address that determines where to downloadit from. When Terraform encounters a new provider in the configuration, it needs to run terraform init first to install the provider plugins in a local directory. Without the plugins, Terraform cannot communicate with the provider and perform the desired actions. Reference = [Provider Requirements], [Provider Installation]

QUESTION NO: 25
Exhibit:
variable "sizes" {
type = list(string)
description = "Valid server sizes"
default = ["small", "medium", "large"]
}
A variable declaration is shown in the exhibit. Which is the correct way to get the value of medium from this variable?
A. var.sizes[0]
B. var.sizes[1]
C. var.sizes[2]
D. var.sizes[3]
Answer: B
Explanation:
Rationale for Correct answer: Terraform lists are zero-indexed. Given ["small", "medium", "large"], index 0 is small, index 1 is medium, and index 2 is large. Therefore, medium is var.sizes[1].
Analysis of Incorrect Options (Distractors):
A: Index 0 returns small, not medium.
C: Index 2 returns large, not medium.
D: Index 3 is out of range for a 3-element list.
Key Concept: Accessing list elements with zero-based indexing in Terraform expressions.
Reference:

QUESTION NO: 26
Terraform providers are always installed from the Internet.
A. True
B. False
Answer: B
Explanation:
Terraform providers are not always installed from the Internet. There are other ways to install provider plugins, such as from a local mirror or cache, from a local filesystem directory, or from a network filesystem. These methods can be useful for offline or air-gapped environments, or for customizing the installation process. You can configure the provider

installation methods using the provider_installation block in the CLI configuration file.

QUESTION NO: 27
A Terraform provider is NOT responsible for:
A. Exposing resources and data sources based on an APUI
B. Managing actions to take based on resources differences
C. Understanding API interactions with some service
D. Provisioning infrastructure in multiple
Answer: D
Explanation:
This is not a responsibility of a Terraform provider, as it does not make sense grammatically or logically. A Terraform provider is responsible for exposing resources and data sources based on an API, managing actions to take based on resource differences, and understanding API interactions with some service.

QUESTION NO: 28
What task does the terraform import command perform?
A. Imports resources from one Terraform state file to another.
B. Imports existing resources into Terraform's state file.
C. Imports a new Terraform module into Terraform's state file.
D. Imports all infrastructure from the configured cloud provider.
E. Imports provider configuration from one state file to another.
Answer: B
Explanation:
Rationale for Correct Answer (B):
terraform import allows Terraform to associate existing resources (created outside of Terraform or by another tool) with a Terraform configuration by writing them into the state file. After import, Terraform can manage those resources.
Analysis of Incorrect Options:
A . From one state file to another: Incorrect, import does not transfer between state files.
C . Importing a module: Incorrect, modules are defined in configuration, not imported.
D . Import all infrastructure: Incorrect, import is per-resource, not bulk.
E . Provider configuration transfer: Incorrect, provider configs are in .tf files, not imported with this command.
Key Concept:
The terraform import command bridges existing resources with Terraform state management.
Reference:
Terraform Exam Objective - Implement and Maintain State.

QUESTION NO: 29
Which of these statements about Terraform Cloud workspaces is false?
A. They have role-based access controls
B. You must use the CLI to switch between workspaces
C. Plans and applies can be triggered via version control system integrations

D. They can securely store cloud credentials

Answer: B

Explanation:

The statement that you must use the CLI to switch between workspaces is false. Terraform Cloud workspaces are different from Terraform CLI workspaces. Terraform Cloud workspaces are required and represent all of the collections of infrastructure in an organization. They are also a major component of role-based access in Terraform Cloud. You can grant individual users and user groupspermissions for one or more workspaces that dictate whether they can manage variables, perform runs, etc. You can create, view, and switch between Terraform Cloud workspaces using the Terraform Cloud UI, the Workspaces API, or the Terraform Enterprise Provider5. Terraform CLI workspaces are optional and allow you to create multiple distinct instances of a single configuration within one working directory. They are useful for creating disposable environments for testing or experimenting without affecting your main or production environment. You can create, view, and switch between Terraform CLI workspaces using the terraform workspace command6. The other statements about Terraform Cloud workspaces are true. They have role-based access controls that allow you to assign permissions to users and teams based on their roles and responsibilities. You can create and manage roles using the Teams API or the Terraform Enterprise Provider7. Plans and applies can be triggered via version control system integrations that allow you to link your Terraform Cloud workspaces to your VCS repositories. You can configure VCS settings, webhooks, and branch tracking to automate your Terraform Cloud workflow8. They can securely store cloud credentials as sensitive variables that are encrypted at rest and only decrypted when needed. You can manage variables using the Terraform Cloud UI, the Variables API, or the Terraform Enterprise Provider9. Reference = [Workspaces]5, [Terraform CLI Workspaces]6, [Teams and Organizations]7, [VCS Integration]8, [Variables]9

QUESTION NO: 30

What does this code do?

terraform { required_providers { aws = ">= 3.0" }}

A. Requires any version of the AWS provider > = 3.0 and <4.0

B. Requires any version of the AWS provider >= 3.0

C. Requires any version of the AWS provider > = 3.0 major release. like 4.1

D. Requires any version of the AWS provider > 3.0

Answer: A

Explanation:

From the Terraform Provider Requirements:

">= 3.0" means any versiongreater than or equal to 3.0- including 4.x and beyond.

A (wrong): Would need >= 3.0, < 4.0

C/D (wrong): > excludes 3.0 - not what's written.